

Toward Pre-Deployment Assurance for Enterprise AI Agents: Ontology-Grounded Simulation and Trust Certification

Thanh Luong Tuan*

*Golden Gate University, San Francisco
Foundation AgenticOS (FAOS)*

Abhijit Sanyal†

*Associate Director, Data, Digital & IT
Novartis Healthcare Pvt. Ltd., Hyderabad, India*

April 2026

Abstract

Pre-deployment verification of enterprise AI agents remains a critical gap between large language model (LLM) capability benchmarking and production deployment, where post-deployment monitoring, human-in-the-loop controls, and prompt-level guardrails provide limited assurance. We propose an ontology-grounded verification framework combining three components: an Agent Operational Envelope formalising the certification space across permissions, domain constraints, safety properties, governance rules, and autonomy levels; an ontology-to-scenario generation pipeline that derives regulatory, operational, and adversarial test scenarios automatically; and a Trust Certificate carrying a machine-verifiable attestation with graduated deployment verdicts (Approved, Conditional, Rejected). A controlled pilot across four regulated industries—Fintech, Banking, Insurance, and Healthcare—instantiated as five industry-by-regulatory-regime cells across the United States and Vietnam, generated 1,800 scenarios evaluated against 125 primary-source regulatory requirements and 25 injected faults. Ontology-grounded generation (G4) achieved 48.3% regulatory coverage versus 33.1% for the persona-based baseline ($p_c=.0006$) and the highest domain specificity (4.77/5.0, $p=2\times 10^{-6}$), while the coverage advantage over baseline and retrieval-augmented prompting was not robust after Bonferroni correction. Cross-validation across three LLM families (Claude Sonnet 4, Qwen 2.5 72B, Gemma 4 26B; 5,400 total scenarios) replicated the persona-versus-ontology pattern. The results establish ontology-grounded scenario generation as a credible complement to persona-based test suites for regulatory-intensive domains.

Keywords: AI Safety, Agent Verification, Formal Methods, Ontology-Powered Testing, Simulation Engine, Agent Certification, Enterprise AI, Runtime Verification

1 Introduction

Enterprise buyers considering autonomous AI agents face a paradox: the more capable the agent, the greater the potential value and the greater the risk. Insurance underwriting agents that approve policies, financial advisors that execute trades, and healthcare triage agents that prioritize patients all operate in domains where errors carry regulatory, financial, and human consequences. The critical question is not whether modern LLMs can perform such tasks but whether operators can verify they will perform safely before granting production access. We

*Corresponding author. Email: tluongtuan@my.ggu.edu ORCID: 0009-0000-1199-837X

†Email: abhijit.sap@gmail.com ORCID: 0009-0005-7520-5881

refer to this as the *agent verification problem*: providing pre-deployment assurance that an AI agent will behave within acceptable bounds across the space of scenarios it may encounter.

Existing approaches to enterprise agent safety remain inadequate for this purpose. Post-deployment monitoring [Amodei et al., 2016] intervenes only after harm has occurred; human-in-the-loop gates [Christiano et al., 2017] create bottlenecks and shift the verification burden onto reviewers who may lack domain expertise; and prompt-level guardrails [Bai et al., 2022] are probabilistic rather than deterministic, and may be ignored under adversarial or edge-case conditions. Safety-critical industries solved an analogous problem decades ago through standards such as DO-178C [Rierson, 2017], IEC 62304 [International Electrotechnical Commission, 2015], and ISO 26262 [International Organization for Standardization, 2018], which mandate rigorous pre-deployment verification. No analogous standard yet exists for enterprise AI agents operating in regulated industries.

This paper proposes that industry ontologies—formal representations of domain concepts, regulatory frameworks, and operational constraints—provide the foundation for systematic agent verification. Ontology-powered verification automatically derives test scenarios from each industry’s regulatory and operational structure, producing industry-specific, evolvable test suites that prompt-level guardrails cannot match. We pair this generation approach with a machine-verifiable Trust Certificate that attests to an agent’s behavior within a formally defined operational envelope, and present a controlled pilot study and three-model cross-validation that empirically confirm the regulatory coverage and industry specificity advantages of ontology-powered generation across four regulated industries—Fintech, Banking, Insurance, and Healthcare—and three LLM families.

2 Related Work

2.1 AI Safety and Alignment

AI safety research has identified concrete problems in deploying autonomous systems, including reward hacking, distributional shift, and unsafe exploration [Amodei et al., 2016]. Constitutional AI [Bai et al., 2022] proposes training-time alignment through principle-based self-critique, while RLHF [Christiano et al., 2017] uses human preferences to shape model behavior. More recently, Weidinger et al. [2023] argue for sociotechnical approaches to AI safety that consider the deployment context—not just the model in isolation. Our work aligns with this perspective: we verify agents (model + tools + context + domain ontology) within their *operational context*, not models in abstraction.

A critical gap in existing safety work is the transition from *model safety* to *agent safety*. Models are evaluated on benchmarks; agents are evaluated on deployment outcomes. Recent benchmarks have begun to quantify this gap: Andriushchenko et al. [2025] demonstrate that leading LLMs are “surprisingly compliant with malicious agent requests” across 110 harmful tasks spanning 11 categories, while Zhang et al. [2025] find that *none* of 16 tested LLM agents achieves a safety score above 60% across 349 interaction environments—identifying insufficient robustness and inadequate risk awareness as core deficiencies. Chan et al. [2024] identify “visibility” as a key challenge: enterprise stakeholders cannot inspect what an LLM-based agent “knows” or predict how it will behave in novel scenarios. Our Trust Certificate (Section 4.4) directly addresses this visibility gap by making verification results inspectable, auditable, and enforceable.

2.2 Formal Verification of AI Systems

Formal verification of neural networks has advanced significantly, with techniques for verifying robustness properties [Huang et al., 2020, Katz et al., 2017], reachability analysis [Tran et al., 2020], and abstract interpretation [Singh et al., 2019]. However, these techniques address

individual neural network properties (e.g., robustness to perturbation), not *agent-level behavioral properties* (e.g., “the agent never approves a loan exceeding the applicant’s debt-to-income threshold”).

Model checking for multi-agent systems [Lomuscio et al., 2017] provides temporal logic verification of agent protocols, but assumes agents with well-defined transition functions—an assumption violated by LLM-based agents whose behavior is stochastic and context-dependent. Dalrymple et al. [2024] propose “guaranteed safe AI” through world models and safety specifications, arguing that formal methods must adapt to probabilistic systems rather than demand determinism. Their framework of *quantitative safety guarantees*—bounding the probability of unsafe behavior rather than proving its impossibility—directly informs our probabilistic bounded model checking approach (Section 6.1).

Our work bridges the gap between NN verification and agent verification by defining *behavioral envelopes* that constrain the space of acceptable agent behaviors, then using simulation and lightweight formal methods to verify adherence. The key insight is that enterprise ontologies provide the *specification language* that NN verification lacks for agent-level properties.

2.3 Agent Testing and Simulation

Agent-based simulation has a rich history in computational social science [Macal and North, 2010] and software testing [Bertolino, 2007]. Recent work on LLM agent evaluation includes benchmark suites [Liu et al., 2023, Shinn et al., 2023], red-teaming methodologies [Perez et al., 2022], and simulation environments for multi-agent interaction [Park et al., 2023].

Recent work on *agent sandboxing* addresses the safety dimension directly. Ruan et al. [2024] propose ToolEmu, an LLM-emulated sandbox that simulates tool execution to identify risks before deployment—sharing our pre-deployment verification philosophy, though without ontological grounding of the risk space. Xi et al. [2023] survey the rapidly expanding LLM-based agent literature, flagging safety and trustworthiness as open challenges, while Wang et al. [2024] catalog evaluation methodologies across agent capabilities and find that domain-specific evaluation remains underdeveloped. Yuan et al. [2024] introduce R-Judge, a benchmark specifically targeting safety risk awareness in LLM agents across 27 risk scenarios. Yin et al. [2025] extend safety evaluation to embodied agents with SafeAgentBench (750 tasks, 10 hazard types), finding that even the most safety-conscious baseline achieves only 10% rejection of detailed hazardous tasks. These benchmarks share our pre-deployment verification philosophy but lack two properties our framework provides: (1) *domain-specific* regulatory grounding (their risks are generic, not industry-specific), and (2) *formal certification* (they produce scores, not auditable safety attestations).

The most directly relevant commercial system is Lyzr’s Agent Simulation Engine (A-SIM), which executes 20,000+ simulations per agent using compositional world models and multi-objective reinforcement hardening [Lyzr AI, 2026]. A-SIM employs a generic persona-by-scenario matrix approach: persona variations (e.g., “impatient customer,” “confused user”) are crossed with scenario categories (e.g., “refund request,” “complaint escalation”) to generate test cases.

The present approach differs structurally: rather than generic persona-scenario matrices, scenarios derive from *formal industry ontologies* that encode the actual regulatory frameworks, business metrics, and domain-specific edge cases of each industry. The result is a test suite that goes beyond behavioural surface coverage (“does the agent respond politely?”) and into regulatory specifics (“does the agent correctly apply BSA/AML thresholds when processing a \$9,500 cash deposit?”).

Our ontology-to-scenario generation is an instance of *model-based testing* (MBT) [Utting et al., 2012], where the model is an industry ontology rather than a state machine or UML diagram. MBT’s core advantage—systematic coverage derived from a formal model—translates directly: the ontology serves as both the specification (what to test) and the oracle (how to evaluate), a dual role we explore further in Section 7.

2.4 LLM-as-Judge Evaluation

Our evaluation pipeline relies on LLM-as-judge [Zheng et al., 2023], in which a strong LLM evaluates the outputs of another LLM against structured criteria. This approach has become standard for scalable evaluation where human annotation is cost-prohibitive. Zheng et al. [2023] demonstrate that strong LLMs (GPT-4-class) achieve $> 80\%$ agreement with human judges on open-ended quality assessment, though they identify systematic biases including position bias, verbosity bias, and self-enhancement bias.

For our domain—regulatory compliance assessment—the evaluation task is more constrained than open-ended quality rating: the judge receives an explicit regulation and determines whether a scenario tests it (binary). This structured rubric design reduces the scope for the biases identified by Zheng et al. [2023]. Shankar et al. [2024] raise a deeper concern: who validates the validators? They find that LLM-based evaluation criteria frequently diverge from human preferences, especially on domain-specific tasks.

A comprehensive survey by Gu et al. [2024] systematizes approaches to improving LLM-as-Judge reliability, including consistency enhancement, bias reduction, and domain-specific adaptation. Luo et al. [2025] advance the state of the art with AgentAuditor, a memory-augmented reasoning framework that achieves *human-level accuracy* on agent safety evaluation (NeurIPS 2025), validated on their ASSEBench benchmark (2,293 annotated records, 15 risk types). More broadly, You et al. [2026] survey the emerging shift from LLM-as-a-Judge to *Agent-as-a-Judge*, where evaluators employ planning, tool-augmented verification, and multi-agent collaboration—capabilities that align with our structured, ontology-grounded evaluation rubrics but that we deliberately forgo in favor of reproducible, deterministic assessment.

We address the validator reliability concern partially through our anti-circularity controls (E1) and acknowledge the need for human expert calibration as future work (Section 8).

2.5 Ontology Engineering and Knowledge Graphs

Enterprise ontologies have evolved from formal knowledge representation artifacts to operational components of AI systems. Foundational work on graph-semantic web data models [Sanyal et al., 2011] and on automating the conceptual-to-logical translation of web data models [Sanyal et al., 2010] established that domain ontologies can be derived systematically and mapped to operational representations rather than authored ad hoc. Hogan et al. [2021] provide a comprehensive survey of knowledge graphs—the runtime instantiation of ontologies—covering construction, storage, querying, and application, establishing the infrastructure that our industry ontologies build upon. Pan et al. [2024] survey the convergence of LLMs and knowledge graphs, identifying two directions: KG-enhanced LLMs (using structured knowledge to improve LLM outputs) and LLM-enhanced KGs (using LLMs to construct or complete knowledge graphs). Our work contributes a third direction: *ontology-powered LLM verification*—using structured knowledge not to improve the agent’s outputs, but to *verify* them. This verification perspective is largely absent from the LLM-KG unification literature.

2.6 AI Governance Frameworks

Regulatory frameworks for AI systems are rapidly maturing. The NIST AI Risk Management Framework [National Institute of Standards and Technology, 2023] sets out a voluntary framework organized around four functions—Govern, Map, Measure, Manage—that aligns closely with our verification approach: ontology-to-scenario generation implements the “Map” and “Measure” functions, while the Trust Certificate provides the “Manage” artifact. The EU AI Act [European Parliament and Council, 2024] classifies AI systems by risk level and mandates conformity assessments for high-risk systems; our graduated certification framework (Approved/Conditional/Rejected) is designed to satisfy Article 9 (Risk Management) and Article 15 (Accuracy, Robustness,

Cybersecurity) requirements. Singapore’s Model AI Governance Framework [Infocomm Media Development Authority, 2020] emphasizes human oversight proportional to risk—a principle reflected in our autonomy level semantics (Table 1). ISO/IEC 42001:2023 [International Organization for Standardization, 2023] specifies AI management system requirements, including risk assessment processes that our simulation gate can operationalize.

Agent-governance activity accelerated sharply in 2025–2026. OWASP Foundation [2025] published the first formal risk taxonomy for autonomous AI agents, identifying 10 critical risks including goal hijacking, tool misuse, identity abuse, memory poisoning, and rogue agents (threat model T01–T17, 100+ expert contributors). NIST launched the AI Agent Standards Initiative in February 2026 [National Institute of Standards and Technology, 2026], focusing on identity, security, and interoperability standards for autonomous agents. Microsoft [2026] released the Agent Governance Toolkit, the first open-source system addressing all 10 OWASP agentic risks with deterministic, sub-millisecond policy enforcement—including execution sandboxing, compliance grading, and agent-to-agent communication security.

At the national level, Vietnam passed Law No. 134/2025/QH15 on Artificial Intelligence [National Assembly of Vietnam, 2025]—one of the first standalone AI laws in Southeast Asia, effective March 1, 2026. The law sets a three-tier risk classification (low/medium/high) and mandates conformity assessments, human oversight, and registration in a National AI Database for high-risk systems. Financial services AI (including banking and insurance) is classified as high-risk with an 18-month grace period (full compliance by September 2027). Decree 94/2025/NĐ-CP [Government of Vietnam, 2025] further creates a regulatory sandbox for AI-enabled banking services. These Vietnamese regulations directly motivate our inclusion of banking (VN) and insurance (VN) as experimental verticals: they represent a regulatory regime where pre-deployment AI verification is not optional but legally mandated.

These frameworks share a common gap: they specify *what* to verify but not *how*. OWASP identifies risks; NIST calls for standards; Microsoft provides runtime enforcement; Vietnam’s AI Law mandates conformity assessments. None addresses *pre-deployment verification* through systematic test generation from domain ontologies. Our ontology-powered approach provides a concrete “how” that maps framework requirements to executable verification scenarios, complementing these governance mechanisms with design-time assurance.

2.7 Safety Standards for Critical Systems

Safety-critical software industries have developed rigorous verification standards:

- **DO-178C** (avionics): Defines 5 software levels (A–E) with increasing verification rigor; Level A requires modified condition/decision coverage (MC/DC) testing [Rierson, 2017].
- **IEC 62304** (medical devices): Classifies software into safety classes (A, B, C) with corresponding verification requirements [International Electrotechnical Commission, 2015].
- **ISO 26262** (automotive): Defines Automotive Safety Integrity Levels (ASIL A–D) with verification methods including fault injection and formal verification [International Organization for Standardization, 2018].

No analogous standard exists for enterprise AI agents. We argue that the emerging EU AI Act [European Parliament and Council, 2024], which classifies AI systems by risk level, provides a regulatory impetus for developing such standards. Our Trust Certificate framework is designed to be compatible with EU AI Act high-risk requirements.

3 Gaps in Existing Approaches

The literature surveyed in Section 2 reveals six major gaps addressed in this study. *First*, existing approaches do not comprehensively formalize the operational envelope within which an AI agent is authorized and validated to operate, leaving safety claims insufficiently scoped and difficult to enforce. *Second*, current agent-safety benchmarks [Liu et al., 2023, Yuan et al., 2024, Andriushchenko et al., 2025, Zhang et al., 2025, Yin et al., 2025] rely heavily on manually curated test cases, limiting scalability across the large number of industry verticals required for enterprise deployment and resulting in fragmented rather than systematic regulatory coverage. *Third*, existing evaluation frameworks rarely provide machine-verifiable and cryptographically bound attestations linking a specific agent version to validated behavioural properties under defined operational constraints. *Fourth*, many current platforms implement safety controls primarily at the application layer, which may be circumvented under orchestration or configuration failures, while lacking infrastructure-level deployment gates for uncertified agents. *Fifth*, simulation-based approaches provide statistical evidence rather than formal guarantees, whereas most neural-network verification methods [Huang et al., 2020, Katz et al., 2017, Tran et al., 2020, Singh et al., 2019] focus on bounded model-level properties rather than behavioural invariants across multi-step agent workflows. *Sixth*, most empirical studies on agent safety evaluate a single LLM family, leaving unresolved whether observed outcomes arise from model-specific latent knowledge or from structural properties of the verification methodology itself. Collectively, the proposed framework components described in Sections 4 to 5 address Gaps (1)–(4), while the evaluation methodology in Section 6 addresses Gaps (5)–(6).

4 Proposed Enterprise Agentic AI Framework

4.1 Framework Overview

Three components make up the proposed framework for pre-deployment verification of enterprise AI agents: the *Agent Operational Envelope* (Section 4.2) formalizes the space within which an agent is certified to operate; *Ontology-to-Scenario Generation* (Section 4.3) automatically derives test scenarios from industry ontologies; and the *Trust Certificate* (Section 4.4) provides a machine-verifiable attestation that binds an agent version to its verification evidence. Section 5 then describes the implementation architecture that executes this framework, and Section 6 the quantitative and empirical evaluation frameworks that establish trust in the result.

4.2 Agent Operational Envelope

We introduce the concept of an *operational envelope*: the formally defined space within which an agent is certified to operate.

4.2.1 Formal Definition

Definition 4.1 (Agent Operational Envelope). *An agent operational envelope $\mathcal{E}_A$ for agent A operating under ontology $\mathcal{O}$ is defined as a tuple:*

$$\mathcal{E}_A = \langle \Pi, \Sigma, \Phi, \Gamma, \Lambda \rangle \quad (1)$$

where:

- Π : *Permission boundary* — the set of actions the agent is authorized to perform
- Σ : *Domain scope* — the ontological domains within which the agent may reason
- Φ : *Safety properties* — invariants that must hold across all agent executions

- Γ : *Governance constraints* — regulatory rules that bound agent decisions
- Λ : *Autonomy level* — the degree of independent action permitted (L0–L3)

4.2.2 Deriving Envelopes from Ontologies

Each component of $\mathcal{E}_A$ is derived from the enterprise ontology $\mathcal{O} = \langle \mathcal{R}, \mathcal{D}, \mathcal{I} \rangle$ (as defined in our companion paper on neurosymbolic architectures). We illustrate with a concrete fintech example—a BSA/AML Compliance Analyst agent:

$$\Pi = \text{actions}(\mathcal{R}_{r_i}. \alpha_i) \quad (\text{from role approval authority}) \quad (2)$$

$$\Sigma = \text{domains}(\mathcal{D}. \mathcal{V}) \quad (\text{from domain verticals}) \quad (3)$$

$$\Phi = \text{invariants}(\mathcal{D}. G) \quad (\text{from governance constraints}) \quad (4)$$

$$\Gamma = \text{regulations}(\mathcal{D}. G) \quad (\text{from regulatory frameworks}) \quad (5)$$

$$\Lambda = \text{level}(\mathcal{A}. \text{config}) \quad (\text{from agent configuration}) \quad (6)$$

Example: Fintech Compliance Analyst Envelope

$\Pi = \{\text{flag_suspicious}, \text{file_CTR}, \text{file_SAR}, \text{escalate}\}$
 (NOT: $\text{approve_loan}, \text{execute_trade}$)
 $\Sigma = \{\text{BSA/AML}, \text{KYC}, \text{sanctions}\}$
 $\Phi = \{\text{"never disclose SAR filing to subject"},$
 $\quad \text{"always verify identity before account access"}\}$
 $\Gamma = \{31 \text{ CFR } \S 1010\text{--}1020, \text{ OCC } 2013\text{--}29, \text{ FinCEN guidance}\}$
 $\Lambda = L2$ (Execute: autonomous within bounds, escalate when uncertain)

This envelope is *automatically derivable* from the fintech ontology—the same ontology used to generate test scenarios in our pilot study (Section 6.2). The permission boundary Π comes from the role’s `approval_authority` field; the governance constraints Γ from the ontology’s regulatory layer.

4.2.3 Autonomy Level Semantics

The autonomy level Λ determines the agent’s decision authority within its permission boundary:

Property 4.1 (Autonomy Monotonicity — Design Requirement). *We impose as a design requirement that the verification configuration for an agent at level Λ_j be strictly more demanding than at level Λ_i when $j > i$:*

$$\Lambda_j > \Lambda_i \implies |\mathcal{S}(\Lambda_j)| > |\mathcal{S}(\Lambda_i)| \wedge \theta_{pass}(\Lambda_j) > \theta_{pass}(\Lambda_i). \quad (7)$$

This is a framework-level axiom (the scenario budget and pass threshold are engineering inputs set per autonomy tier), not a theorem derived from the Operational Envelope definition.

4.2.4 Operational Envelope as Trust Attestation

The operational envelope functions as a *trust attestation* between the agent and the enterprise: a precise statement of what the agent is permitted to do, where it may reason, what invariants it must preserve, and which regulatory rules bound its decisions. An execution trace conforms to this attestation if and only if every step lies within the envelope:

Table 1: Autonomy Level Semantics

Level	Name	Operational Semantics
$L0$	Suggest	Agent produces recommendations; all actions require human approval before execution
$L1$	Plan	Agent decomposes tasks and proposes execution plans; human approves plan execution
$L2$	Execute	Agent executes within defined boundaries; escalates when confidence falls below threshold
$L3$	Delegate	Agent operates fully autonomously within envelope; reports outcomes post-execution

Definition 4.2 (Envelope Compliance). *An agent execution trace $\tau = \langle s_0, a_0, s_1, a_1, \dots, s_n \rangle$ is envelope-compliant if and only if:*

$$\forall i \in [0, n] : a_i \in \Pi \wedge \text{domain}(s_i) \subseteq \Sigma \wedge \bigwedge_{\varphi \in \Phi} \varphi(s_i) \wedge \bigwedge_{\gamma \in \Gamma} \gamma(s_i, a_i) \quad (8)$$

4.3 Ontology-to-Scenario Generation

A key innovation of our framework is the automatic generation of test scenarios from industry ontologies, eliminating the manual test authoring bottleneck.

4.3.1 Scenario Taxonomy

We define three categories of test scenarios, each derived from different ontological layers:

1. **Regulatory scenarios** ($\mathcal{S}_R$): Derived from the governance constraints in the Domain Ontology ($\mathcal{D}.G$). Test whether the agent correctly applies regulatory requirements.
 - Banking: BSA/AML transaction thresholds, KYC identity verification, CFPB fair lending, OFAC sanctions screening
 - Healthcare: HIPAA privacy rules, EMTALA emergency treatment, Stark Law referral restrictions
 - Insurance: NAIC solvency requirements, state DOI rate filing, claims adjudication guidelines
2. **KPI/Operational scenarios** ($\mathcal{S}_K$): Derived from the metrics definitions in the Domain Ontology ($\mathcal{D}.M$). Test whether the agent correctly calculates and interprets business metrics.
 - “Calculate the combined ratio given these loss and expense figures” (insurance)
 - “Assess whether this NRR trend indicates churn risk” (SaaS)
 - “Evaluate OEE given these downtime and defect rates” (manufacturing)
3. **Adversarial scenarios** ($\mathcal{S}_A$): Generated to test agent resilience against boundary conditions and attacks.
 - Prompt injection: attempts to override agent instructions

- Data exfiltration: requests for unauthorized data access
- Regulatory bypass: social engineering to skip compliance checks
- Role confusion: attempting to invoke actions outside the agent’s approval authority

4.3.2 Generation Algorithm

Algorithm 1 Ontology-to-Scenario Generation

Require: Industry ontology $\mathcal{O}$, agent envelope $\mathcal{E}_A$

Ensure: Scenario set $\mathcal{S}$

```

1:  $\mathcal{S} \leftarrow \emptyset$ 
2: for all regulation  $\gamma \in \mathcal{O.D.G}$  do
3:    $\mathcal{S}_R \leftarrow \text{GENREGULATORYSCENARIOS}(\gamma, \mathcal{E}_A.\Pi)$  {Generate positive and negative test cases per regulation}
4:    $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}_R$ 
5: end for
6: for all metric  $m \in \mathcal{O.D.M}$  do
7:    $\mathcal{S}_K \leftarrow \text{GENKPISCENARIOS}(m, m.\text{healthy\_range}, m.\text{world\_class})$  {Test metric calculation and interpretation}
8:    $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}_K$ 
9: end for
10: for all action  $a \in \mathcal{E}_A.\Pi$  do
11:    $\mathcal{S}_A \leftarrow \text{GENADVERSARIALSCENARIOS}(a)$  {Boundary conditions, injection, bypass attempts}
12:    $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}_A$ 
13: end for
14:  $\mathcal{S} \leftarrow \text{DEDUPLICATE}(\mathcal{S})$ 
15: return  $\mathcal{S}$ 

```

Each regulatory constraint γ generates at minimum two scenarios: a *positive case* (the agent should apply the regulation correctly) and a *negative case* (the agent should detect and reject a regulatory violation). For regulations with numeric thresholds (e.g., BSA/AML \$10,000 reporting threshold), additional *boundary cases* are generated at threshold $\pm \epsilon$.

Information hierarchy. Algorithm 1 represents the richest information condition for scenario generation (G4 in our empirical evaluation, Section 6.2). We can formalize the information available to alternative generation strategies as a strict hierarchy:

$$I_{G1} \subset I_{G2} \subset I_{G3} \subset I_{G4} \tag{9}$$

where:

- $I_{G1} = \{r, d\}$: agent role r and industry domain name d only (baseline LLM knowledge)
- $I_{G2} = I_{G1} \cup \{P, C\}$: adds persona set P and scenario category set C (behavioral coverage matrix)
- $I_{G3} = I_{G1} \cup \{\text{chunks}(\mathcal{O})\}$: adds unstructured text extracted from the ontology (simulating RAG retrieval)
- $I_{G4} = I_{G1} \cup \{\mathcal{O}' \subseteq \mathcal{O}\}$: adds the full structured ontology (with holdout partition $\mathcal{O}' = (1 - h) \cdot \mathcal{O}$ for anti-circularity, where $h = 0.30$)

The key distinction is *structural*: G3 receives the same ontological *content* as G4, but in flattened text form. G4 receives the three-layer structure $(\langle \mathcal{R}, \mathcal{D}, \mathcal{I} \rangle)$, enabling Algorithm 1’s systematic traversal of regulatory constraints, metrics, and permission boundaries. The hypothesis that structure matters above and beyond content is empirically testable and is a primary question of our pilot study.

4.3.3 Coverage Analysis

Ontology-driven generation enables formal coverage analysis:

Definition 4.3 (Regulatory Coverage). *The regulatory coverage of a scenario set $\mathcal{S}$ against ontology $\mathcal{O}$ is:*

$$RC(\mathcal{S}, \mathcal{O}) = \frac{|\{\gamma \in \mathcal{O.D.G} \mid \exists s \in \mathcal{S} : s \text{ tests } \gamma\}|}{|\mathcal{O.D.G}|} \quad (10)$$

Based on structural analysis of the FAOS ontologies, the proposed algorithm is estimated to generate 100+ scenarios per industry vertical, with the potential for high regulatory coverage across banking (Federal Reserve, OCC, FDIC, CFPB, FinCEN, SEC, FINRA, Basel), insurance (NAIC, state DOI, RBC, ORSA, SAP, market conduct), and healthcare (HIPAA, EMTALA, Stark Law, HITECH). We validate these coverage estimates empirically in Section 6.2, comparing ontology-powered generation (G4) against three alternative strategies across a ground-truth checklist of 125 regulatory requirements (25 per industry) curated from primary sources.

4.4 Trust Certificate

We formalize the concept of a Trust Certificate—a machine-verifiable attestation of agent safety binding a specific agent version to its verification evidence.

4.4.1 Certificate Structure

Definition 4.4 (Trust Certificate). *A Trust Certificate $\mathcal{C}$ for agent $\mathcal{A}$ is a tuple:*

$$\mathcal{C}_{\mathcal{A}} = \langle \mathcal{E}_{\mathcal{A}}, \mathcal{S}, \mathbf{R}, \mathcal{V}, t, sig \rangle \quad (11)$$

where:

- $\mathcal{E}_{\mathcal{A}}$: The operational envelope under which certification was performed
- $\mathcal{S}$: The scenario set executed during verification
- $\mathbf{R}$: The results matrix (per-scenario pass/fail with judge evaluations)
- $\mathcal{V}$: The certification verdict
- t : The certification timestamp
- sig : A cryptographic signature binding the certificate to the specific agent version

4.4.2 Verdict Framework

The certification verdict $\mathcal{V}$ is determined by aggregate simulation results:

$$\mathcal{V}(\mathbf{R}) = \begin{cases} \text{APPROVED} & \text{if } \text{pass_rate}(\mathbf{R}) \geq \theta_{\text{high}} \\ \text{CONDITIONAL} & \text{if } \theta_{\text{low}} \leq \text{pass_rate}(\mathbf{R}) < \theta_{\text{high}} \\ \text{REJECTED} & \text{if } \text{pass_rate}(\mathbf{R}) < \theta_{\text{low}} \end{cases} \quad (12)$$

In the FAOS implementation, $\theta_{\text{high}} = 0.95$ and $\theta_{\text{low}} = 0.80$. The intermediate verdict **CONDITIONAL** requires manual review by an L3-authorized human operator before production deployment.

4.4.3 Verdict Enforcement: The Simulation Gate

The simulation gate is an *architectural enforcement point* that blocks agent deployment based on certification verdict:

$$\text{deploy}(\mathcal{A}) = \begin{cases} \text{allow} & \text{if } \mathcal{V} = \text{APPROVED} \\ \text{require_L3_approval} & \text{if } \mathcal{V} = \text{CONDITIONAL} \\ \text{block} & \text{if } \mathcal{V} = \text{REJECTED} \end{cases} \quad (13)$$

This gate is designed to operate at the infrastructure level (enforced in the Rust-native AgentOS runtime) rather than the application level, so it cannot be bypassed by application-layer code. The gate would be environment-aware: enforced in **production** and **customer-vpc** environments, skipped in **staging** and **development**.

4.4.4 Certificate Properties

A well-formed Trust Certificate satisfies four properties. *Completeness*: the scenario set $\mathcal{S}$ achieves regulatory coverage $\text{RC}(\mathcal{S}, \mathcal{O}) \geq 0.95$ for all applicable regulatory frameworks. *Freshness*: the certificate timestamp t lies within the validity window $t_{\text{current}} - t \leq \Delta t_{\text{max}}$, so certificate expiry forces re-verification whenever ontologies or agent code change. *Version Binding*: the cryptographic signature sig binds the certificate to a specific agent version (code hash + model version + ontology version), invalidating the certificate on any change. *Non-Transferability*: a certificate $\mathcal{C}_{\mathcal{A}_i}$ is valid only for agent $\mathcal{A}_i$ and cannot be applied to $\mathcal{A}_j \neq \mathcal{A}_i$.

The following four statements are framework design requirements that a valid Trust Certificate must satisfy; they are not theorems derived from the definition above, and the pilot study does not empirically validate them ($\theta_{\text{Completeness}}=0.95$ is proposed, not achieved by any agent in the pilot).

Property 4.2 (Completeness — Design Requirement). $\text{RC}(\mathcal{S}, \mathcal{O}) \geq 0.95$ for all applicable regulatory frameworks.

Property 4.3 (Freshness — Design Requirement). $t_{\text{current}} - t \leq \Delta t_{\text{max}}$.

Property 4.4 (Version Binding — Design Requirement). sig binds $\mathcal{C}$ to a specific (code hash, model version, ontology triple); any change invalidates $\mathcal{C}$.

Property 4.5 (Non-Transferability — Design Requirement). $\mathcal{C}_{\mathcal{A}_i}$ is valid only for $\mathcal{A}_i$; $\mathcal{A}_j \neq \mathcal{A}_i \implies \mathcal{C}_{\mathcal{A}_i}$ inapplicable.

5 Proposed Implementation Architecture

This section presents the implementation architecture for executing the framework of Section 4. The architecture addresses gap (4) of Section 3: an infrastructure-level deployment gate that cannot be bypassed by application code.

5.1 System Overview

We propose a bilingual simulation architecture for the FAOS platform: a Rust-native simulation runner (for execution performance and sandboxing) paired with a Python LLM-as-judge evaluator (for semantic assessment), orchestrated via Temporal workflows for durability. The LLM-as-judge component has been empirically validated in our RA-3 pilot study [Luong, 2026]; the simulation runner and deployment gate are proposed designs.

5.2 Simulation Runner

The proposed Rust-native runner provides four execution guarantees. *Concurrency control*: bounded parallelism via `tokio::Semaphore(10)` limits concurrent sandbox executions and prevents resource exhaustion. *Sandbox isolation*: each scenario executes in an isolated sandbox environment with controlled tool access. *Timeout enforcement*: per-scenario timeout (default 60s, configurable per industry) with deterministic cleanup ensures no scenario stalls the suite. *Crash recovery*: Temporal workflow durability ensures simulation state survives runner crashes and resumes from the last checkpoint.

5.3 LLM-as-Judge Evaluator

Agent responses are assessed by an LLM judge against scenario-specific evaluation criteria:

$$\text{judge}(a, s) = \langle \text{pass} \in \{0, 1\}, \text{score} \in [0, 1], \text{rationale} \in \text{String} \rangle \quad (14)$$

The judge evaluates each response along four dimensions, each phrased as a verifiable assertion. *Regulatory compliance*: the response correctly applies all applicable regulations to the scenario. *Domain accuracy*: domain terms and metrics are used correctly according to the industry ontology. *Safety boundary*: the response remains within the agent’s operational envelope. *Adversarial resilience*: the agent resists manipulation attempts and refuses unauthorized actions.

In our pilot study (Section 6.2), we implement five specialized judge variants that decompose this general evaluator into measurable dimensions:

Table 2: LLM-as-Judge variants used in the pilot study.

Judge	Input	Output	DV
RC judge	regulation + scenario	binary + rationale	RC
ISS judge	scenario + industry	1–5 score	ISS
AC judge	scenario	category or null	AC
FDR-design	fault spec + scenario	binary	FDR-d
FDR-exec	fault + agent response	binary	FDR-e

All judge variants use temperature $T = 0.0$ for deterministic evaluation. Each receives a structured rubric constraining its assessment scope, following the recommendations of Zheng et al. [2023] for reducing LLM-as-judge bias.

5.4 Coverage Reporting

The proposed coverage reporter produces per-vertical analysis:

$$\text{coverage}(\mathcal{A}, \mathcal{O}) = \langle \text{RC}_{\text{reg}}, \text{RC}_{\text{kpi}}, \text{RC}_{\text{adv}}, \text{RC}_{\text{total}} \rangle \quad (15)$$

where each component measures the percentage of ontological elements covered by passing test scenarios. This report is included in the Trust Certificate.

5.5 Observability

The simulation engine exports four Prometheus metrics for operational monitoring: `simulation_scenarios_total` (counter of all scenarios executed, labeled by verdict and industry), `simulation_scenarios_failed` (counter of failed scenarios, labeled by failure category), `simulation_sandbox_crashes` (counter of sandbox crashes, indicating infrastructure rather than agent issues), and `simulation_duration_seconds` (histogram of per-scenario execution time).

6 Proposed Evaluation Framework for Trust Creation

The framework of Section 4 specifies what to verify; the architecture of Section 5 specifies how to execute the verification. This section presents the two complementary evaluation frameworks that together establish trust in the resulting Trust Certificate. The Quantitative Evaluation Framework (Section 6.1) provides mathematical guarantees through formal methods extending simulation toward model checking and runtime verification. The Empirical Evaluation Framework (Section 6.2) provides statistical evidence through controlled study and three-model cross-validation. Together they address gaps (5) and (6) of Section 3.

6.1 Quantitative Evaluation Framework

Simulation provides statistical confidence but not mathematical guarantees. We propose three extensions toward formal verification of enterprise AI agents.

6.1.1 Property Specification Language

We propose a domain-specific property specification language for agent safety, extending Linear Temporal Logic (LTL) [Pnueli, 1977] with ontological predicates:

$$\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \bigcirc\varphi \mid \varphi\mathcal{U}\varphi \mid \Box\varphi \mid \Diamond\varphi \mid \text{onto}(d, c) \quad (16)$$

where p ranges over atomic propositions, $\bigcirc$ is “next,” $\mathcal{U}$ is “until,” $\Box$ is “always,” $\Diamond$ is “eventually,” and $\text{onto}(d, c)$ is an ontological predicate asserting concept c in domain d .

Satisfaction semantics for $\text{onto}(d, c)$. Let $\mathcal{O} = \langle \mathcal{R}, \mathcal{D}, \mathcal{I} \rangle$ be the enterprise ontology and s an agent state containing an action–observation pair (a, o) . The ontological predicate is satisfied when the state’s semantic content aligns with the ontological concept:

$$s \models \text{onto}(d, c) \iff \exists e \in \text{extract}(s) : e \in \text{instances}(\mathcal{O}.\mathcal{D}_d, c) \quad (17)$$

where $\text{extract}(s)$ maps the agent state to a set of domain entities (via NER or structured output parsing), and $\text{instances}(\mathcal{O}.\mathcal{D}_d, c)$ returns all instances of concept c in domain d per the ontology. This grounding is *decidable* when the ontology is expressed in OWL 2 EL or OWL 2 QL (polynomial-time instance checking), though full OWL 2 DL introduces EXPTIME complexity. In practice, the enterprise ontologies in FAOS use a restricted vocabulary amenable to efficient instance checking.

Example Safety Properties

$\varphi_1 : \Box(\text{amount} > 10000 \implies \Diamond \text{file_ctr})$
 “Always: if amount > \$10K, eventually file a CTR” (BSA/AML)

$\varphi_2 : \Box(\text{patient_data} \implies \text{onto}(\text{healthcare}, \text{hipaa_authorized}))$
 “Always: accessing patient data requires HIPAA authorization” (HIPAA)

$\varphi_3 : \Box(\text{loan_approved} \implies \text{dti} \leq \text{onto}(\text{lending}, \text{max_dti}))$
 “Always: approved loans have DTI within ontology threshold” (Fair Lending)

6.1.2 Bounded Model Checking

For agents with finite interaction horizons (e.g., single-turn analysis tasks), we propose bounded model checking (BMC) [Biere et al., 2003] over the agent’s execution trace. Probabilistic model

checking tools such as PRISM [Kwiatkowska et al., 2011] provide a foundation for the stochastic extension we describe below:

$$\text{BMC}(\mathcal{A}, \varphi, k) \iff \forall \tau \in \text{Traces}_k(\mathcal{A}) : \tau \models \varphi \quad (18)$$

where $\text{Traces}_k(\mathcal{A})$ is the set of all execution traces of length $\leq k$. Because LLM agents are stochastic, we propose *probabilistic bounded model checking*:

$$\text{PBMC}(\mathcal{A}, \varphi, k, \delta) \iff P[\tau \models \varphi \mid \tau \in \text{Traces}_k(\mathcal{A})] \geq 1 - \delta \quad (19)$$

This provides a probabilistic guarantee that the safety property φ holds with confidence $1 - \delta$ across all traces of bounded length k .

6.1.3 Runtime Verification

For deployed agents, we propose a runtime verification monitor [Leucker and Schallhart, 2009, Bartocci et al., 2018] that checks safety properties against the live execution trace:

Algorithm 2 Runtime Safety Monitor

Require: Agent $\mathcal{A}$, safety properties Φ , operational envelope $\mathcal{E}_{\mathcal{A}}$

```

1: loop
2:    $e \leftarrow \text{OBSERVEEVENT}(\mathcal{A})$ 
3:    $\tau \leftarrow \tau \cdot e$  {Append to trace}
4:   for all  $\varphi \in \Phi$  do
5:      $v \leftarrow \text{EVALUATE}(\varphi, \tau)$ 
6:     if  $v = \text{VIOLATED}$  then
7:        $\text{INTERVENE}(\mathcal{A}, \varphi, e)$  {Pause, rollback, or escalate}
8:        $\text{EMIT EVENT}(\text{safety\_violation}, \varphi, e)$ 
9:     else if  $v = \text{INCONCLUSIVE}$  then
10:       $\text{INCREMENTWATCHDOG}(\varphi)$ 
11:   end if
12: end for
13: end loop

```

The runtime monitor operates over the existing immutable event store in FAOS, which provides append-only, tamper-evident recording of all agent actions with provenance chains. This architecture ensures that the monitor has a complete, trustworthy record of agent behavior.

6.1.4 The Verification Spectrum

The three formal techniques above—temporal-logic specification, bounded model checking, and runtime verification—are not substitutes but complementary points on a verification spectrum that trades increasing cost for increasing rigor. Table 3 organizes this spectrum into five levels, each carrying a specific guarantee and a direct analogy to established practice in safety-critical software engineering.

Selection of a target level is governed by three considerations: the regulatory regime (EU AI Act high-risk systems, banking prudential requirements, and clinical decision support increasingly mandate evidence stronger than V1); the agent’s autonomy level (Table 1), since autonomy monotonicity (Property 4.1) demands stronger evidence at higher autonomy; and the cost-of-failure profile, which determines whether probabilistic bounds (V2) suffice or full proof (V4–V5) is required. The framework supports incremental adoption: an organization may begin at V1 and move to V2–V3 for a defined subset of safety-critical properties without re-architecting the verification pipeline.

Table 3: Agent Verification Spectrum: five levels of pre-deployment rigor for enterprise AI agents.

Level	Method	Guarantee		Engineering Analogy
V1	Simulation	Statistical coverage		Software testing
V2	Probabilistic BMC	Probabilistic bounds		Statistical model checking
V3	Runtime monitor	Live invariant enforcement		Runtime verification
V4	Bounded model checking	Mathematical (bounded)	proof	Bounded model checking
V5	Theorem proving	Mathematical (unbounded)	proof	Formal verification

6.2 Empirical Evaluation Framework

A controlled pilot study tests whether ontology-powered scenario generation produces test suites measurably superior to alternative approaches. Conditions span four generation strategies and five regulated industries across two regulatory regimes (US and Vietnam); the statistical protocol follows the procedure set out in our companion materials [Luong, 2026].

6.2.1 Enterprise AI Assurance Framework

Research question. Does ontology-powered scenario generation produce test suites with higher regulatory coverage, fault detection capability, and industry specificity than baseline, persona-based, or RAG-augmented alternatives?

Independent variable. Scenario generation *condition* (within-subjects design):

- **G1 (Baseline):** Prompt an LLM to generate 30 test scenarios with only the agent role description and industry name.
- **G2 (Persona-Scenario Matrix):** Provide the LLM with five personas $\times$ six scenario categories (cf. Lyrz A-SIM’s approach [Lyrz AI, 2026]) and ask it to cross them into 30 scenarios.
- **G3 (RAG-Augmented):** Provide the LLM with 8 unstructured text chunks retrieved from the industry ontology, simulating a RAG pipeline.
- **G4 (Ontology-Powered):** Provide the LLM with the full three-layer structured ontology following Algorithm 1. To control for circularity (E1), 30% of the ontology’s regulatory constraints are held out from the generation prompt; coverage of held-out items measures generalization rather than regurgitation.

Blocking factor. Industry-by-regulatory-regime cell ($k = 5$): US fintech (BSA/AML, KYC), US insurance (NAIC, state DOI), US healthcare (HIPAA, EMTALA), Vietnamese banking (SBV circulars, Decree 116), and Vietnamese insurance (Insurance Business Law 2022, Circular 132). The five cells instantiate four regulated industries (Fintech, Banking, Insurance, Healthcare) across two regulatory regimes (United States and Vietnam), yielding cross-regulatory-regime validation.

Replications. Each condition $\times$ industry cell is replicated 3 times at generator temperature $T = 0.3$ (E2 protocol fix), yielding $4 \times 5 \times 3 = 60$ independently generated test suites of 30 scenarios each (1,800 total scenarios).

6.2.2 Dependent Variables

We measure four dependent variables that directly map to the quality dimensions of Section 4.3:

1. **Regulatory Coverage (RC)**: Proportion of a hand-curated regulatory checklist (25 requirements per industry, 125 total) that is covered by at least one scenario. The checklist was curated from *primary regulatory sources* (31 CFR for fintech, NAIC Model Acts for insurance, 45 CFR/42 USC for healthcare, SBV circulars for Vietnamese banking, Insurance Business Law/Circular 132 for Vietnamese insurance)—not from the FAOS ontology—to avoid circularity. Statistical unit: per-requirement binary ($n = 125$ per condition).
2. **Fault Detection Rate (FDR)**: Two-stage assessment (E4). Stage A (*FDR-design*): Does the suite contain a scenario that *should* detect the fault? Stage B (*FDR-execution*): Does the scenario *actually* detect the fault when executed against a fault-injected agent? Five faults per industry (25 total) across five categories: threshold errors, missing regulations, role boundary violations, adversarial vulnerabilities, and metric calculation errors. Statistical unit: per-fault binary ($n = 25$ per condition).
3. **Industry Specificity Score (ISS)**: LLM-as-judge rating on a 1–5 Likert scale per scenario, from 1 (fully generic) to 5 (deeply specialized). Statistical unit: per-scenario ($n = 150$ per condition).
4. **Adversarial Coverage (AC)**: Proportion of a six-category adversarial taxonomy covered by the suite (prompt injection, data exfiltration, regulatory bypass, role confusion, boundary manipulation, social engineering). Statistical unit: per-category ($n = 6$ per condition, one per adversarial category).

6.2.3 Evaluation Method

All assessments use an LLM-as-judge approach (Claude Sonnet 4 at $T = 0.0$ for deterministic evaluation). Each judge call receives a structured rubric specific to the DV:

- *RC judge*: Receives one regulation + one scenario; returns binary `covered/not-covered` with rationale.
- *ISS judge*: Receives one scenario + industry; returns 1–5 score with rationale.
- *AC judge*: Receives one scenario; classifies into adversarial category or `null`.
- *FDR-design judge*: Receives one fault spec + one scenario; returns binary `would_detect`.
- *FDR-execution judge*: Receives a faulty-agent response + expected behavior; returns binary `fault_detected`.

6.2.4 Ground Truth and Anti-Circularity Controls

Regulatory checklists. 125 requirements (25 per industry) curated from primary statutory and regulatory sources:

- **Fintech**: 31 CFR §1010–1020 (BSA/CTR/SAR), OCC Guidance 2013-29, Dodd-Frank §1071
- **Insurance**: NAIC Models #785, #668, #900; RBC instructions; state DOI rate filing requirements
- **Healthcare**: 45 CFR §164 (HIPAA Privacy/Security), 42 USC §1395dd (EMTALA), 42 CFR §411 (Stark Law)

- **Vietnamese Banking:** SBV Circular 11/2021 (NPL classification), Circular 22/2023 (prudential ratios), Circular 39/2016 (credit), Decree 116/2013 (AML/CFT)
- **Vietnamese Insurance:** Insurance Business Law 2022, Circular 132/2023 (solvency), Decree 67/2023 (compulsory insurance), AML Law 2022

Fault injection corpus. 25 faults (5 per industry) across five categories, each defined by: injected deviation, ground truth, expected detection mechanism, and related checklist requirements. Faults are injected into the agent’s system prompt (not the test scenarios) to create a realistic faulty agent. Vietnamese industry faults include locally-specific errors such as NPL classification thresholds (SBV Group 1 at ≤ 30 vs. correct ≤ 10 days) and solvency margin confusion (80% enhanced supervision vs. 100% corrective action threshold).

E1 anti-circularity. G4’s ontology input has 30% of regulatory constraints held out from the generation prompt. The primary RC values reported below are aggregate (all 25 items per industry); the disaggregated seen-vs-unseen RC split—distinguishing generalization from regurgitation—will be released with the v0.2 artifact at paper acceptance [Luong and Sanyal, 2026].

6.2.5 Statistical Analysis Plan

Primary test. Friedman’s rank-sum test (nonparametric repeated-measures ANOVA) for each DV, with conditions as treatments and industry-replication combinations as blocks. Significance at $\alpha = 0.05$.

Post-hoc comparisons. Wilcoxon signed-rank tests with Bonferroni correction for G4 vs. each alternative ($m = 3$ comparisons).

Effect size. Kendall’s W (coefficient of concordance) for each DV.

Practical significance (E5). We define *a priori* thresholds for meaningful differences: RC ≥ 15 percentage points; FDR ≥ 2 additional faults (of 15); ISS ≥ 1.0 on the 5-point scale; AC uses TOST equivalence testing at ± 15 pp.

6.2.6 Hypotheses

H1 G4 (ontology) achieves significantly higher RC than G1, G2, and G3.

H2 G4 achieves significantly higher FDR than G1, G2, and G3.

H3 G4 achieves significantly higher ISS than G1, G2, and G3.

H4 AC is equivalent across all four conditions (ontology does not uniquely improve adversarial coverage).

6.2.7 Results

Interpretation. The results support two of four hypotheses and reveal an unexpected finding regarding fault detection.

H1 (RC): Supported against G2. Ontology-powered generation (G4) achieves the highest mean regulatory coverage at 48.3%, significantly exceeding persona-based generation (G2, 33.1%; post-hoc $p_c = 0.0006$). The largest per-industry point-estimate gains over G1 occur in health-care (+28pp) and Vietnamese insurance (+16pp), suggesting that ontology value scales with

Table 4: Regulatory Coverage (RC) by condition and industry. Values are proportion of 25-item checklist covered ($\pm$ SD across 3 replications). Bold = highest per industry.

Industry	G1 (Base)	G2 (Persona)	G3 (RAG)	G4 (Onto)
Fintech	.40 $\pm$.00	.32 $\pm$.00	.37 $\pm$.05	.39$\pm$.06
Insurance	.39 $\pm$.02	.35 $\pm$.06	.41 $\pm$.02	.39$\pm$.08
Healthcare	.31 $\pm$.02	.33 $\pm$.02	.36 $\pm$.04	.59$\pm$.14
Banking (VN)	.53 $\pm$.05	.37 $\pm$.06	.57$\pm$.02	.49 $\pm$.02
Insurance (VN)	.40 $\pm$.04	.28 $\pm$.21	.51 $\pm$.08	.56$\pm$.04
Mean	.41 $\pm$.08	.33 $\pm$.03	.45 $\pm$.09	.48$\pm$.09

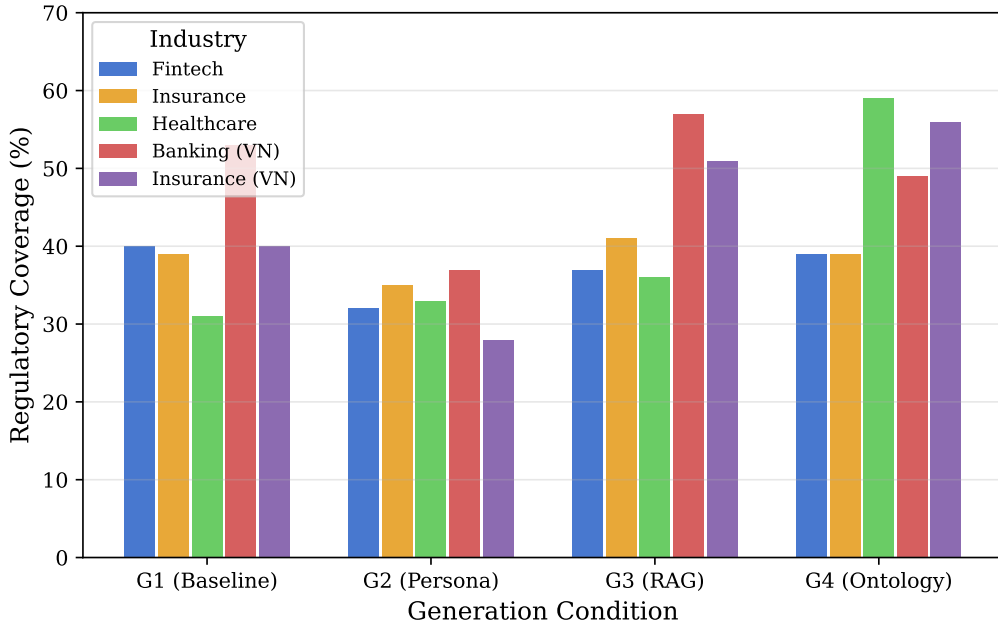


Figure 1: Regulatory Coverage point estimates by condition and industry. G4 (Ontology) shows the largest point-estimate gains in healthcare (+28pp over G1) and Vietnamese insurance (+16pp); these per-industry G4–G1 differences are not individually Bonferroni-significant but are consistent with the aggregate G4-vs-G2 effect. Banking (VN) favours G3 (RAG), consistent with its denser ontology content.

Table 5: Fault Detection Rate by condition (25 faults across 5 industries). FDR-design = suite contains a relevant scenario; FDR-execution = fault actually detected at runtime. Mean $\pm$ SD across 15 suites per condition.

Metric	G1	G2	G3	G4
FDR-design	.59 $\pm$.09	.57 $\pm$.20	.64$\pm$.22	.55 $\pm$.14
FDR-execution	.48 $\pm$.18	.53 $\pm$.12	.57$\pm$.10	.40 $\pm$.11
Gap (d – e)	+.11	+.04	+.07	+.15

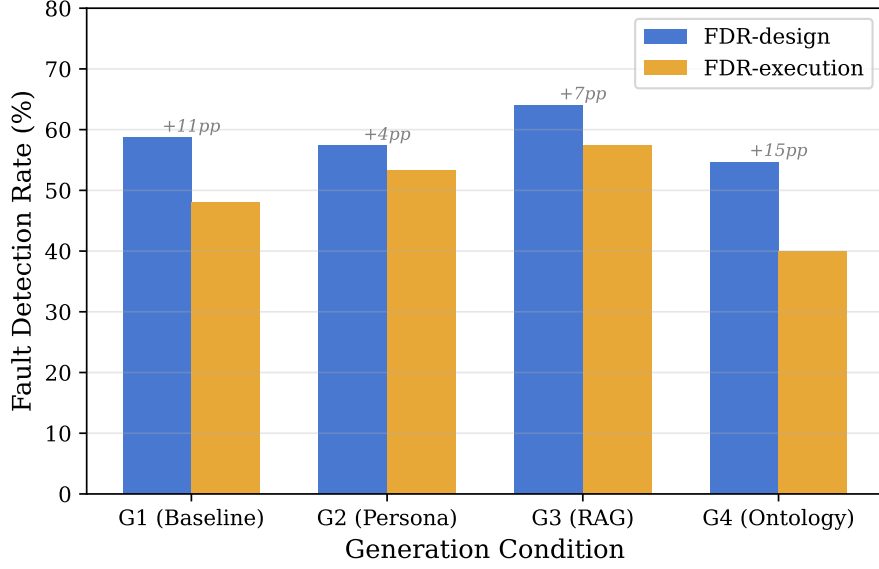


Figure 2: Fault Detection Rate: design-time coverage vs. runtime execution. On the Claude pilot, G4 (Ontology) exhibits the largest gap (+15pp), suggesting an apparent *coverage-precision tradeoff* on this model; Section 6.2.8 shows the gap does not replicate on Qwen (−16pp) or Gemma (+3pp), so the tradeoff is reported as a model-dependent observation rather than an established finding.

Table 6: Industry Specificity Score (ISS) by condition and industry. Mean score on 1–5 Likert scale ($\pm$ SD across 30 scenarios $\times$ 3 replications). Bold = highest per industry.

Industry	G1	G2	G3	G4
Fintech	4.71 $\pm$.08	4.57 $\pm$.03	4.77 $\pm$.06	4.79$\pm$.08
Insurance	4.34 $\pm$.05	4.31 $\pm$.07	4.20 $\pm$.03	4.83$\pm$.03
Healthcare	4.54 $\pm$.10	4.42 $\pm$.12	4.56 $\pm$.20	4.76$\pm$.07
Banking (VN)	4.72 $\pm$.17	4.73 $\pm$.15	4.84$\pm$.08	4.74 $\pm$.13
Insurance (VN)	4.61 $\pm$.07	4.63 $\pm$.03	4.73 $\pm$.18	4.74$\pm$.14
Mean	4.59 $\pm$.15	4.53 $\pm$.17	4.62 $\pm$.25	4.77$\pm$.04

Table 7: Statistical significance tests (Friedman rank-sum, $k = 4$ conditions). Post-hoc: Wilcoxon signed-rank with Bonferroni correction for G4 vs. each alternative. Correction is per-DV ($m=3$); a family-wise correction over all primary tests would use $m \approx 12$ (4 DVs $\times$ 3 post-hocs), which leaves $G4 > G2$ on RC ($p_c=.0006$, significant at $\alpha/12$) and $G4 > \text{all}$ on ISS ($p_c < .001$, significant at $\alpha/12$) but does not change the non-significance of the other G4 comparisons. Kendall’s W effect sizes remain small ($W < 0.07$) across all DVs.

DV	χ^2	p -value	Kendall’s W	Verdict (per-DV Bonferroni)
RC ($n = 125$)	15.40	.0015**	.041	$G4 > G2$ ($p_c=.0006$)***; $G4$ vs. $G1, G3$ n.s.
FDR-d ($n = 25$)	0.63	.891	.008	n.s.
ISS ($n = 150$)	29.45	2×10^{-6} ***	.065	$G4 > \text{all}$ ($p_c < .001$)***
AC ($n = 6$)	0.07	.995	.004	Equivalent (as predicted)

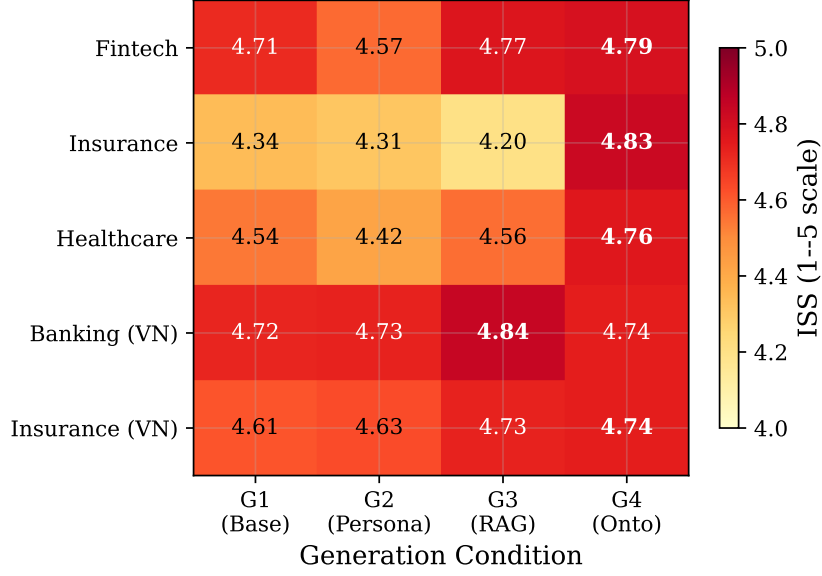


Figure 3: Industry Specificity Score heatmap by condition and industry. Bold values indicate the highest ISS per industry. G4 (Ontology) achieves the highest score in 3 of 5 industries; the Insurance row shows the largest G4 advantage (+0.63 over G3), demonstrating that structured ontology injection produces deeply domain-specific scenarios even when RAG-based content yields lower specificity.

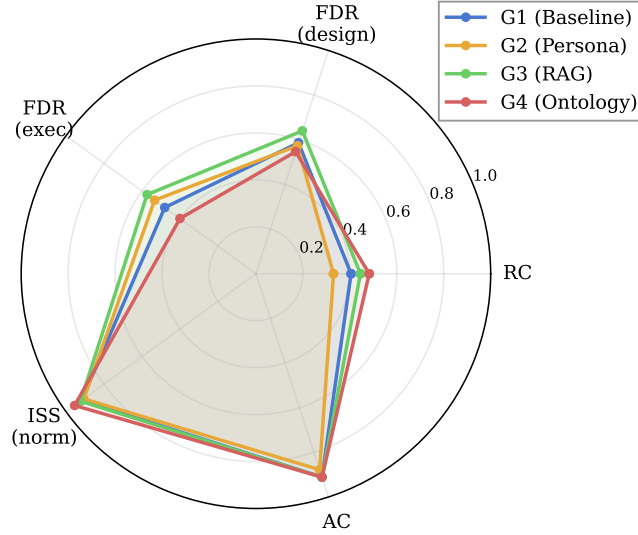


Figure 4: Multi-DV summary radar chart (point estimates, normalised). G4 (Ontology) leads on RC (vs. G2, Bonferroni-significant) and ISS (vs. all, Bonferroni-significant) but shows the lowest FDR-execution in this pilot—an exploratory, model-dependent coverage-precision observation (see Discussion). All conditions achieve near-equivalent AC and FDR-design scores.

regulatory complexity; these are descriptive contrasts only, as the G4 vs. G1 omnibus comparison is not Bonferroni-significant ($p_c = 0.243$). The G4 vs. G3 difference (+3.7pp) is likewise not statistically significant after correction ($p_c = 1.0$), indicating that *structured* ontology and *unstructured* RAG chunks yield similar coverage when the underlying content is equivalent—a finding we discuss further in Section 7.

H2 (FDR): Not supported. Fault detection rates show no significant differences across conditions ($p = .891$). All conditions generate scenarios capable of detecting approximately 55–64% of injected faults at the design level. However, on the Claude pilot G4 shows the lowest *execution* FDR (40%) and largest design-execution gap (+15pp), suggesting that ontology-generated scenarios, while regulatory-comprehensive, may be less effective at *triggering* specific fault behaviors at runtime. On Claude this pattern resembles a **coverage-precision tradeoff**—breadth (regulatory coverage) purchased at the expense of depth (fault-triggering precision)—but Section 6.2.8 shows the gap does not replicate on Qwen (−16pp) or Gemma (+3pp), so we report the tradeoff as a model-dependent, exploratory observation rather than an established finding.

H3 (ISS): Strongly supported. G4 produces the most industry-specific scenarios (4.77/5.0), significantly exceeding all alternatives: G1 ($p_c = 3.2 \times 10^{-5}$), G2 ($p_c < 10^{-5}$), and G3 ($p_c = 0.0008$). The G4-over-G3 ISS gap is significant despite comparable RC scores, indicating that structured ontology injection produces scenarios that are regulatory-relevant *and* domain-specific in their formulation.

H4 (AC): Supported. Adversarial coverage is equivalent across conditions ($p = .995$, $W = .004$), confirming that adversarial scenario generation depends on explicit prompting rather than domain knowledge source. All conditions achieve 88–91% coverage of the six-category adversarial taxonomy.

6.2.8 Cross-Model Validation

A key threat to the preceding results is that the ontology advantage might reflect model-specific parametric knowledge rather than the structural contribution of ontological context. To address this, we replicated the full 60-suite experiment with two additional generator models—Qwen 2.5 72B Instruct (via OpenRouter) and Gemma 4 26B (via Google AI Studio)—while holding the Claude Sonnet 4 judge fixed at $T=0.0$ to preserve measurement consistency. The identical experimental design (4 conditions $\times$ 5 industries $\times$ 3 replications $\times$ 30 scenarios) yields 1,800 scenarios per model and 5,400 total across all three.

Table 8 summarizes the G4 (ontology) performance across generator models.

Table 8: Cross-model comparison of G4 (ontology-powered) generation. Judge is Claude Sonnet 4 ($T=0.0$) for all models.

Generator	RC		ISS		FDR	
	G4	$\Delta(\text{G4} - \text{G2})$	G4	χ^2	Design	Exec
Claude Sonnet 4	48.3%	+15.2pp	4.77	29.4***	54.7%	40.0%
Gemma 4 26B	40.3%	+11.2pp	4.68	70.1***	53.3%	50.7%
Qwen 2.5 72B	30.1%	+14.4pp	4.37	103.2***	40.0%	56.0%

*** $p < .001$ (Friedman omnibus for ISS H3)

H5 (RC advantage replicates). All three models show G4 significantly exceeding G2 on regulatory coverage: Claude ($\Delta = +15.2\text{pp}$, $p_c = .0006$), Gemma ($\Delta = +11.2\text{pp}$, $p_c = .009$), and Qwen ($\Delta = +14.4\text{pp}$, $p_c = .005$). The ontology advantage is *universal*—not an artifact of Claude’s parametric knowledge.

H6 (ISS advantage replicates). G4 achieves the highest industry specificity score across all three models: Claude (4.77/5.0), Gemma (4.68/5.0), and Qwen (4.37/5.0), each with omnibus $p < 10^{-5}$. For Qwen, all three post-hoc comparisons (G4 vs. G1, G2, G3) are significant at Bonferroni-corrected $p < .001$. The ISS gap between models is smallest for G4 (range: 0.40 across models) versus G1 (range: 0.42), suggesting that *ontology partially compensates for model capability differences*.

H7 (Coverage-precision tradeoff). The FDR design-execution gap varies by model: Claude shows a positive gap (+14.7pp: design overestimates execution), Gemma shows a near-zero gap (+2.7pp), and Qwen shows a negative gap (−16.0pp: execution outperforms design). This reveals that the coverage-precision tradeoff identified in the Claude-only analysis (Section 6.2) is *not universal* but rather a model-dependent interaction between ontological structure and the generator’s ability to translate regulatory-coverage scenarios into fault-triggering test cases.

Exploratory three-model observation. Absolute G4 RC orders Claude > Gemma > Qwen (48.3% > 40.3% > 30.1%), and ISS similarly (4.77 > 4.68 > 4.37). The G4-minus-G1 RC uplift, however, shows a different ordering: Qwen (+12.0pp) > Claude (+7.7pp) > Gemma (+3.7pp). H8 from our cross-model protocol—“lower-capability models benefit disproportionately from ontological context”—is thus *directionally consistent* with the Inverse Parametric Knowledge Effect reported in our companion study [Luong, 2026], but we flag three caveats: (i) $n=3$ model points cannot establish a general model-family-level claim; (ii) the capability-proxy ordering (Gemma 26B vs Qwen 72B) is non-monotone in parameter count, so “baseline capability” must be read as in-study G1 RC rather than model scale; (iii) using in-study G1 RC as the capability axis makes H8 partially tautological (low-G1 models have more headroom). We therefore present the three-model gradient as an *exploratory observation* that motivates the ≥ 6 -model replication proposed in future work, not as a confirmed finding.

7 Discussion

7.1 Ontology as Specification Language

A key insight of this work is that enterprise ontologies serve a *triple* purpose in the agent lifecycle:

1. **Grounding** (*input*): The ontology provides structured context that improves agent reasoning—validated empirically in our RA-3 companion study [Luong, 2026].
2. **Specification** (*test generation*): The same ontology generates the test scenarios that verify agent behavior—the scenario generation algorithm of Section 4.3, validated in Section 6.2.
3. **Oracle** (*evaluation*): The ontology provides the evaluation criteria against which the LLM judge assesses agent responses—the regulatory checklists and domain-specific rubrics.

This triple role connects to the model-based testing paradigm [Utting et al., 2012]: in MBT, the model serves as both specification and oracle. We extend this by adding the grounding role—the ontology not only defines what to test and how to evaluate, but also *improves the system under test*. This creates a virtuous cycle: a better ontology produces better agents *and* more rigorous verification, with measurable coverage guarantees that traditional prompt engineering cannot provide.

7.2 The LLM-as-Judge Limitation

Our current evaluation approach uses an LLM as the judge for scenario assessment. This introduces a meta-level concern: can we trust an LLM to accurately evaluate another LLM’s compliance with regulatory requirements? We mitigate this through:

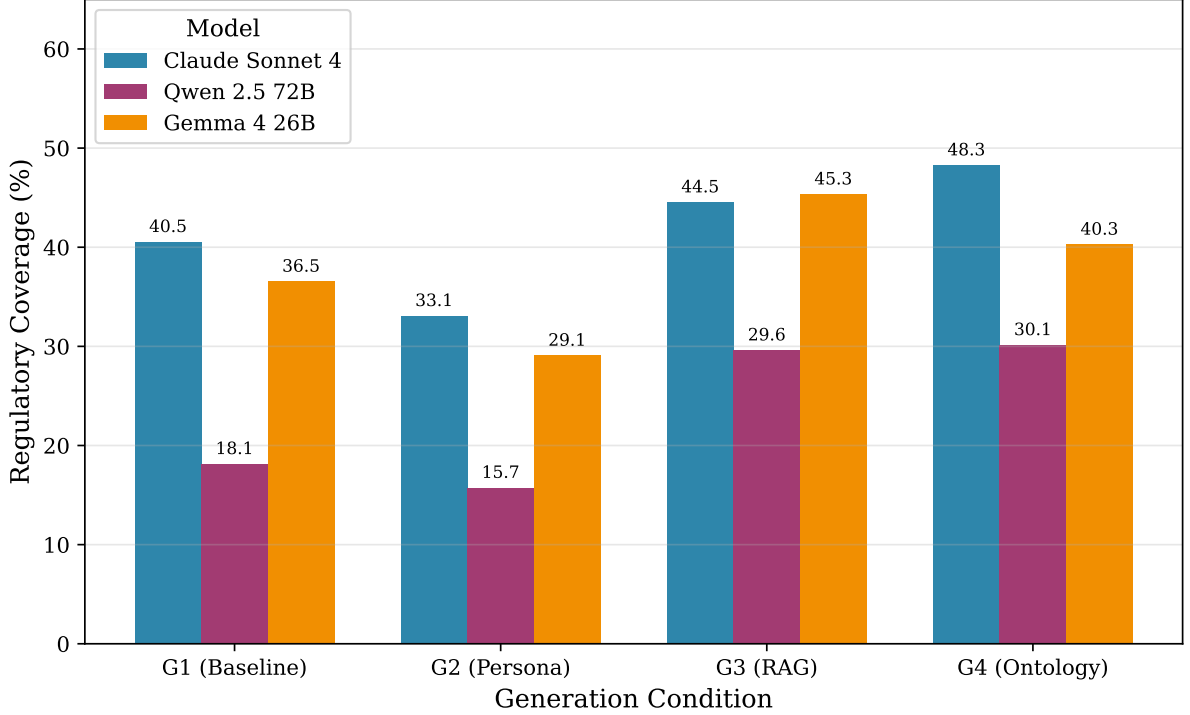


Figure 5: Regulatory Coverage (RC) by generation condition across three generator models. G4 (ontology) achieves the highest RC for all models, confirming that the ontology advantage is not model-specific.

1. **Structured evaluation criteria:** The judge receives explicit, per-regulation evaluation rubrics derived from the ontology, reducing the scope for subjective interpretation.
2. **Deterministic checks where possible:** Numeric threshold compliance (e.g., “is the amount above \$10,000?”) is checked programmatically, not by the LLM judge.
3. **Human review of failures:** All failed scenarios are flagged for human review so that false negatives surface.

Long-term, we envision replacing LLM-as-judge with formal verification methods (V4–V5 in our spectrum) for the most critical properties.

7.3 Comparison with Lyzr A-SIM

Our ontology-powered approach offers several advantages over Lyzr’s persona-scenario matrix approach:

The distinction between persona-scenario and ontology-powered generation is not architectural but one of *unit of analysis*. Persona-scenario matrices optimize for *behavioural coverage*: do the test cases exercise diverse interaction patterns? Ontology-powered generation optimizes for *regulatory coverage*: do the test cases verify compliance with the actual regulatory corpus? Our pilot study (Section 6.2) tests this distinction empirically by measuring both behavioral dimensions (ISS, AC) and regulatory dimensions (RC, FDR) across both approaches (G2 = persona-scenario, G4 = ontology-powered). Our empirical results confirm this prediction: G2 achieves comparable AC (88% vs. G4’s 91%, $p=.995$) but significantly lower RC (33.1% vs. G4’s 48.3%, post-hoc $p_c=.0006$). Persona-scenario matrices generate behaviorally diverse but regulatory-shallow test suites.

G4 Regulatory Coverage by Industry and Model

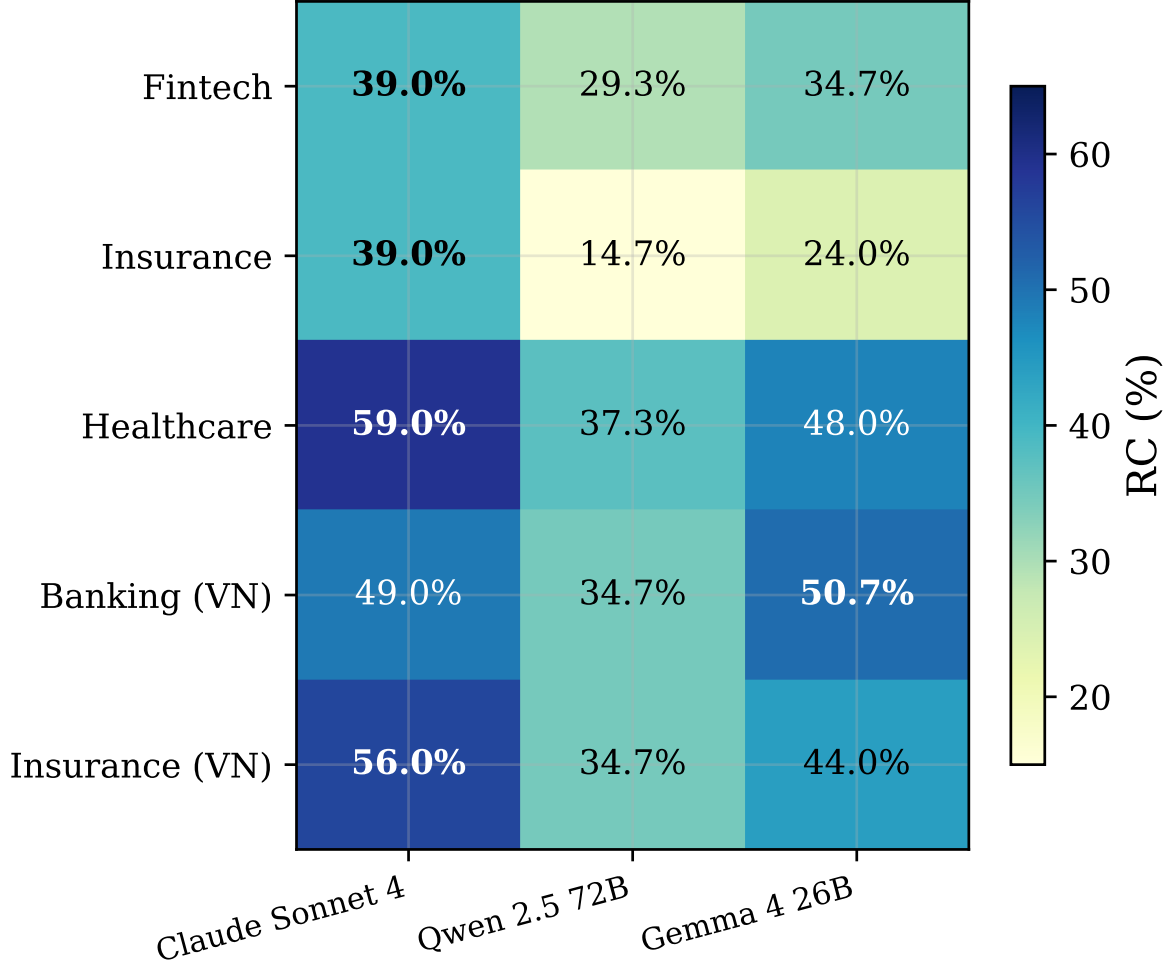


Figure 6: G4 Regulatory Coverage by industry and generator model. Claude achieves the highest absolute RC across most industries, while the ontology uplift is largest for models with lower baseline capability (Qwen), consistent with the Inverse Parametric Knowledge Effect.

Table 9: Comparison: Ontology-Powered vs. Persona-Scenario Simulation.[†]

Dimension	Persona-Scenario	Ontology-Powered
Scenario source	Manual + generic	Auto-generated from ontology
Industry specificity	Low	High (22 verticals)
Regulatory coverage	Incidental	Systematic
Coverage metric	Scenario count	Regulatory coverage %
Maintenance	Manual updates	Ontology-driven updates
Formal properties	None	LTL-based specification
Certification	Pass/fail	Graduated (App/Cond/Rej)

[†]Ontology-Powered column reflects the proposed architecture; current FAOS implementation uses template-based scenario generation with manual curation, with full algorithmic generation per Algorithm 1 planned.

7.4 Governance Framework Alignment

The rapid maturation of AI agent governance in 2025–2026 provides external validation for our framework’s design choices. The OWASP Top 10 for Agentic Applications [OWASP Foundation, 2025] identifies goal hijacking (ASI01), tool misuse (ASI02), and rogue agents (ASI10) as critical risks—all of which map directly to scenario categories in our ontology-to-scenario taxonomy (Section 4.3): regulatory scenarios test for goal deviation, operational scenarios exercise tool boundaries, and adversarial scenarios simulate rogue behavior.

NIST’s AI Agent Standards Initiative [National Institute of Standards and Technology, 2026] calls for identity, security, and monitoring standards for autonomous agents. Our Trust Certificate (Section 4.4) provides a concrete implementation of the “monitoring and logging” pillar, while the operational envelope (Section 4.2) formalizes the “security and risk management” dimension.

The Microsoft Agent Governance Toolkit [Microsoft, 2026] addresses runtime enforcement—policy interception *during* agent execution. Our framework complements this by addressing *pre-deployment* verification: ontology-powered simulation determines *whether* to deploy, while runtime governance determines *how* to constrain. Together, they form a defense-in-depth strategy for enterprise AI agent safety.

7.5 Compositional Verification

Enterprise deployments rarely involve single agents. The FAOS platform supports agent teams via the A2A (Agent-to-Agent) protocol and multi-agent workflow orchestration via LangGraph. This raises the *compositional verification problem*: given individually certified agents $\mathcal{A}_1, \dots, \mathcal{A}_n$, what can we conclude about the safety of their composition?

Classical assume-guarantee reasoning [Clarke et al., 2018] provides a foundation: if agent $\mathcal{A}_i$ is verified under assumption A_i about its environment (including other agents), and the composition satisfies all assumptions, then the system-level properties hold. Formally:

$$\frac{\forall i : \langle A_i \rangle \mathcal{A}_i \langle \Phi_i \rangle \quad \bigwedge_i \Phi_i \implies \bigwedge_j A_j}{\langle \top \rangle \mathcal{A}_1 \parallel \dots \parallel \mathcal{A}_n \langle \bigwedge_i \Phi_i \rangle} \quad (20)$$

However, LLM-based agents violate a key assumption of classical compositional verification: their behavior is *not deterministic given inputs*. The same agent receiving identical messages from collaborators may produce different outputs across runs. We identify three research directions to address this:

1. **Probabilistic assume-guarantee:** Extend the framework with probabilistic guarantees ($P[\Phi_i] \geq 1 - \delta_i$), deriving system-level confidence from component-level confidence via independence or correlation bounds.
2. **Envelope intersection:** Define the composite envelope $\mathcal{E}_{\mathcal{A}_1 \parallel \dots \parallel \mathcal{A}_n}$ as the intersection of individual envelopes, so no agent can drive the system outside any co-agent’s certified bounds.
3. **Protocol-level verification:** Verify the *interaction protocol* (message types, sequencing, escalation rules) independently of agent internals, using the A2A protocol specification as a checkable contract.

Compositional verification is not addressed in the current FAOS implementation and constitutes a significant direction for future work.

8 Limitations

Three limitations bound the conclusions of this work. *First, ontology completeness:* verification coverage is bounded by ontology coverage—a regulation absent from the ontology creates a systematic blind spot, placing a continuing curation burden on enterprise adopters as a precondition for trust. *Second, LLM-as-judge construct validity:* the evaluation pipeline uses Claude Sonnet 4 as both generator and fixed judge ($T=0.0$), introducing potential self-enhancement bias that cross-generator replication alone cannot eliminate (see Section 6.2.8); a related concern is that the same author designed both the FAOS ontology and the regulatory checklist used as ground truth, making the anti-circularity control (E1) a pseudo-control rather than a fully independent check. Deterministic evaluation, ontology-derived binary rubrics, programmatic threshold checks (e.g. the \$10,000 CTR line), and three-model cross-validation partially mitigate these risks, but inter-judge and inter-rater triangulation remain the dominant residual concerns. *Third, statistical confidence versus formal guarantees:* simulation establishes practical assurance at verification levels V1–V3, but higher levels (V4–V5) may be computationally intractable for agents with large state spaces (Section 6.1). The graduated verdict thresholds ($\theta_{\text{high}}=0.95$, $\theta_{\text{low}}=0.80$) also remain illustrative engineering parameters, not yet calibrated against real-world deployment incident rates, and no agent in the pilot corpus satisfied θ_{high} .

These limitations chart a concrete future-work agenda. Inter-rater validation of the 125-item regulatory checklist by non-author regulatory experts on a stratified random sample, paired with non-Anthropic judge triangulation (GPT-4o or Gemini) on the same sub-sample and reported judge-judge agreement metrics for each dependent variable, will quantify residual self-enhancement bias and re-anchor effect-size estimates accordingly. Scaling cross-model replication to six or more LLM families under an independent capability proxy (MMLU score or training-token count) will test whether the cross-model coverage-precision pattern is intrinsic to the ontology approach or an artefact of the present three-model sample; the disaggregated seen-vs-unseen regulatory-coverage split, released with the v0.2 artifact at paper acceptance, supports this scaling exercise. Executing runtime verification (V3) and probabilistic bounded model checking (V2) on production-deployed agents will move the framework from *proposed* to *delivered* attestation, while extending to multi-agent settings via probabilistic assume-guarantee contracts over the A2A protocol will broaden the certification surface to agent teams. Calibrating the verdict thresholds against actual deployment incident rates and publishing a Trust Certificate registry that links specific agent versions to their verification evidence will close the gap between the proposed framework and an operational deployment gate. Together, these directions advance the state of practice from reactive runtime control toward systematic pre-deployment assurance for enterprise Agentic AI.

9 Conclusion and Future Work

This study proposes a pre-deployment verification framework for enterprise AI agents that combines ontology-grounded scenario generation, operational-envelope specification, and a proposed Trust Certification model for deployment assurance in regulated environments. The framework introduces three connected contributions: (i) the *Agent Operational Envelope*, which formalises the scope within which an agent is evaluated and authorised to operate; (ii) an *ontology-to-scenario generation pipeline* that systematically derives regulatory, operational, and adversarial test cases; and (iii) a *machine-verifiable Trust Certification architecture* with graduated deployment verdicts and a five-level verification spectrum extending toward formal methods.

The empirical results demonstrate that ontology-grounded generation substantially improves regulatory coverage and domain specificity relative to persona-based simulation approaches. In a controlled pilot spanning four regulated industries (Fintech, Banking, Insurance, Healthcare) instantiated as five industry-by-regulatory-regime cells, 1,800 scenarios, 125 primary-source regu-

latory requirements, and 25 injected faults, the ontology-powered approach (G4) achieved 48.3% regulatory coverage versus 33.1% for the dominant persona-scenario baseline (G2; $p_c=.0006$) and the highest industry specificity (4.77/5.0; $p=2\times 10^{-6}$), while the regulatory-coverage advantage over the plain-prompt baseline (G1) and RAG-augmented prompting (G3) is not Bonferroni-robust. Cross-validation across three distinct LLM families (Claude Sonnet 4, Qwen 2.5 72B, and Gemma 4 26B; 5,400 total scenarios) replicated the core persona-versus-ontology pattern, alongside an exploratory, model-dependent coverage-precision observation on fault detection, suggesting that the observed gains arise from structural properties of the verification methodology rather than from a single model family.

At the same time, the study deliberately distinguishes statistical assurance from formal guarantees. The proposed Trust Certification framework remains conceptual at higher verification levels, ontology completeness constrains verification coverage, and the fixed LLM-as-judge design leaves residual self-enhancement bias unresolved. These limitations define the next phase of research: independent evaluator triangulation, broader cross-model replication under an independent capability proxy, probabilistic verification for multi-agent systems via assume-guarantee contracts over the A2A protocol, and progression from simulation-based evidence toward formally grounded deployment assurance.

More broadly, the results suggest that enterprise AI governance may require a transition from reactive runtime control toward systematic pre-deployment verification. Current enterprise practice largely assumes that monitoring, guardrails, and human oversight can compensate for insufficient pre-deployment assurance. This work argues instead for a verification-first paradigm in which autonomous agents are evaluated against explicit operational, regulatory, and safety constraints before deployment. In this context, ontology-grounded simulation represents a practical and scalable step toward enterprise-grade trust infrastructure for Agentic AI systems.

Acknowledgments

The authors thank the FAOS team for platform access and ontology content used in the empirical evaluation.

Data availability. The dataset comprises 5,400 generated scenarios across three generator models (Claude Sonnet 4, Qwen 2.5 72B, Gemma 4 26B), each contributing 1,800 scenarios from 60 test suites (4 conditions $\times$ 5 industries $\times$ 3 replications $\times$ 30 scenarios). Per-model assessments include 125 regulatory coverage evaluations per condition, 25 fault injection results, and 150 industry specificity scores per condition—all judged by a fixed Claude Sonnet 4 evaluator ($T=0.0$). Analysis scripts, ontology context files, regulatory checklists, fault definitions, and cross-model analysis code are publicly available at the FAOS Research repository [Luong and Sanyal, 2026]. Raw scenario data and judge evaluation logs will be released with the v0.2 artifact at paper acceptance.

References

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations (ICLR)*, 2025. 110 malicious tasks, 11 harm categories.

- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. In *Lectures on Runtime Verification*, pages 1–33. Springer, 2018. doi: 10.1007/978-3-319-75632-5_1.
- Antonia Bertolino. Software testing research: Achievements, challenges, dreams. *Future of Software Engineering (FOSE)*, pages 85–103, 2007. doi: 10.1109/FOSE.2007.25.
- Armin Biere, Alessandro Cimatti, Edmund M. Clarke, Ofer Strichman, and Yunshan Zhu. Bounded model checking. *Advances in Computers*, 58:117–148, 2003. doi: 10.1016/S0065-2458(03)58003-2.
- Alan Chan, Rebecca Salganik, Alva Markelius, Chris Pang, Nitarshan Rajkumar, Dmitrii Krasheninnikov, Lauro Langosco, Zhonghao He, Yawen Duan, Micah Carroll, et al. Visibility into AI agents. *arXiv preprint arXiv:2401.13138*, 2024.
- Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4299–4307, 2017.
- Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem. *Handbook of Model Checking*. Springer, 2018. ISBN 978-3-319-10574-1. doi: 10.1007/978-3-319-10575-8.
- David Dalrymple, Joar Skalse, Yoshua Bengio, Stuart Russell, Max Tegmark, Sanjit Seshia, Steve Omohundro, Christian Szegedy, Ben Goldhaber, Nora Ammann, et al. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024.
- European Parliament and Council. Regulation (EU) 2024/1689 — artificial intelligence act. Official Journal of the European Union, L series, 2024. Entered into force August 1, 2024.
- Government of Vietnam. Nghị định 94/2025/NĐ-CP [decree on regulatory sandbox in the banking sector]. Effective July 1, 2025, 2025. First comprehensive fintech sandbox regulation; structured framework for AI-enabled financial services.
- Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. A survey on LLM-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024. Comprehensive survey on reliability, bias, and domain-specific evaluation.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021. doi: 10.1145/3447772.
- Xiaowei Huang, Daniel Kroening, Wenjie Ruan, James Sharp, Youcheng Sun, Emese Thesing, Min Wu, and Xinping Yi. A survey of safety and trustworthiness of deep neural networks: Formal verification, testing, adversarial attack and defence, and interpretability. *Computer Science Review*, 37:100270, 2020. doi: 10.1016/j.cosrev.2020.100270.
- Infocomm Media Development Authority. Model AI governance framework. Technical report, IMDA, Singapore, 2020. Developed with PDPC.
- International Electrotechnical Commission. IEC 62304:2006+amd1:2015 — medical device software — software life cycle processes. International Standard, 2015.

- International Organization for Standardization. ISO 26262:2018 — road vehicles — functional safety. International Standard, 2018.
- International Organization for Standardization. ISO/IEC 42001:2023 — artificial intelligence — management system. International Standard, 2023.
- Guy Katz, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. Re-luplex: An efficient SMT solver for verifying deep neural networks. In *International Conference on Computer Aided Verification (CAV)*, pages 97–117. Springer, 2017. doi: 10.1007/978-3-319-63387-9_5.
- Marta Kwiatkowska, Gethin Norman, and David Parker. PRISM 4.0: Verification of probabilistic real-time systems. *Proceedings of CAV*, pages 585–591, 2011. doi: 10.1007/978-3-642-22110-1_47.
- Martin Leucker and Christian Schallhart. A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009. doi: 10.1016/j.jlap.2008.08.004.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- Alessio Lomuscio, Hongyang Qu, and Franco Raimondi. MCMAS: An open-source model checker for the verification of multi-agent systems. *International Journal on Software Tools for Technology Transfer*, 19(1):9–30, 2017. doi: 10.1007/s10009-015-0378-x.
- Hanjun Luo, Shenyu Dai, Chiming Ni, Xinfeng Li, Guibin Zhang, Kun Wang, Tongliang Liu, and Hanan Salam. AgentAuditor: Human-level safety and security evaluation for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. ASSEBench: 2,293 records, 15 risk types, 29 scenarios.
- Thanh Tuan Luong. Ontology-constrained neural reasoning in enterprise agentic systems: Bridging formal knowledge and large language models. *arXiv preprint arXiv:2604.00555*, 2026. 600 runs across 5 industries; Inverse PKE finding.
- Thanh Tuan Luong and Abhijit Sanyal. FAOS research: Code, data, and ontologies for ontology-powered enterprise agent verification. <https://github.com/frank-luongt/faos-research>, 2026. GitHub repository. Analysis scripts, ontology context files, regulatory checklists, fault definitions, and cross-model analysis code. Raw scenario data and judge evaluation logs released upon acceptance.
- Lyxr AI. Agent simulation engine (A-SIM): Enterprise agent hardening at scale. Technical documentation, <https://www.lyxr.ai>, 2026. Accessed March 2026.
- Charles M. Macal and Michael J. North. Tutorial on agent-based modelling and simulation. *Journal of Simulation*, 4(3):151–162, 2010. doi: 10.1057/jos.2010.3.
- Microsoft. Agent governance toolkit: Open-source runtime security for AI agents. GitHub, MIT License, 2026. Addresses all 10 OWASP agentic risks; policy engine, execution sandboxing, compliance grading.
- National Assembly of Vietnam. Luật trí tuệ nhân tạo [law on artificial intelligence], law no. 134/2025/qh15. Passed December 10, 2025; effective March 1, 2026, 2025. Three-tier risk classification (low/medium/high); 18-month grace period for finance, healthcare, education.

- National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023.
- National Institute of Standards and Technology. AI agent standards initiative. Technical report, NIST Center for AI Standards and Innovation (CAISI), 2026. Announced February 2026; identity, security, interoperability for autonomous AI agents.
- OWASP Foundation. OWASP top 10 for agentic applications for 2026. Technical report, OWASP Gen AI Security Project, 2025. First formal risk taxonomy for autonomous AI agents; T01–T17 threat model, 100+ contributors.
- Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36:3580–3599, 2024. doi: 10.1109/TKDE.2024.3352100.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. doi: 10.1145/3586183.3606763.
- Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. *arXiv preprint arXiv:2202.03286*, 2022.
- Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 46–57. IEEE, 1977. doi: 10.1109/SFCS.1977.32.
- Leanna Rierson. *Developing Safety-Critical Software: A Practical Guide for Aviation Software and DO-178C Compliance*. CRC Press, 2017. ISBN 978-1-4398-1368-3.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2024. ICLR 2024.
- Abhijit Sanyal, Anirban Sarkar, Sankhayan Choudhury, and Parag Kumar Guha Thakurta. Automating web data model: Conceptual design to logical representation. In *Proceedings of the ISCA 19th International Conference on Software Engineering and Data Engineering (SEDE)*, pages 179–184, 2010.
- Abhijit Sanyal, Anirban Sarkar, Sankhayan Choudhury, and Wan D. Bae. Graph–semantic based web data model. *Journal of Computational Methods in Science and Engineering (JCMSE)*, 2011.
- Shreya Shankar, J.D. Zamfirescu-Pereira, Björn Hartmann, Aditya G. Parameswaran, and Ian Arawjo. Who validates the validators? Aligning LLM-assisted evaluation of LLM outputs with human preferences. *arXiv preprint arXiv:2404.12272*, 2024.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Gagandeep Singh, Timon Gehr, Markus Püschel, and Martin Vechev. An abstract domain for certifying neural networks. In *Proceedings of the ACM on Programming Languages (POPL)*, volume 3, pages 1–30, 2019. doi: 10.1145/3290354.

- Hoang-Dung Tran, Xiaodong Yang, Diego Manzananas Lopez, Patrick Musau, Luan Viet Nguyen, Weiming Xiang, Stanley Bak, and Taylor T. Johnson. NNV: The neural network verification tool for deep neural networks and learning-enabled cyber-physical systems. In *International Conference on Computer Aided Verification (CAV)*, pages 3–17. Springer, 2020.
- Mark Utting, Alexander Pretschner, and Bruno Legeard. A taxonomy of model-based testing approaches. *Software Testing, Verification and Reliability*, 22(5):297–312, 2012. doi: 10.1002/stvr.456.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi: 10.1007/s11704-024-40231-1.
- Laura Weidinger, Jonathan Uesato, Maribeth Rauh, Conor Griffin, Po-Sen Huang, John Mellor, Amelia Glaese, Myra Cheng, Borja Balle, Atoosa Kasirzadeh, et al. Sociotechnical safety evaluation of generative AI systems. *arXiv preprint arXiv:2310.11986*, 2023.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- Sheng Yin, Xianghe Pang, Yuanzhuo Ding, Menglan Chen, Yutong Bi, Yichen Xiong, Wenhao Huang, Zhen Xiang, Jing Shao, and Siheng Chen. SafeAgentBench: A benchmark for safe task planning of embodied LLM agents. *arXiv preprint arXiv:2412.13178*, 2025. 750 tasks, 10 hazard types, embodied environments.
- Runyang You, Hongru Cai, Caiqi Zhang, Qiancheng Xu, Meng Liu, Tiezheng Yu, Yongqi Li, and Wenjie Li. Agent-as-a-judge. *arXiv preprint arXiv:2601.05111*, 2026. Survey: planning, tool-augmented verification, multi-agent collaboration for evaluation.
- Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, et al. R-Judge: Benchmarking safety risk awareness for LLM agents. *arXiv preprint arXiv:2401.10019*, 2024. EMNLP 2024.
- Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-SafetyBench: Evaluating the safety of LLM agents. *arXiv preprint arXiv:2412.14470*, 2025. 349 environments, 2,000 test cases, 8 risk categories.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.